Section 1 — Incident Header

Incident ID: INC-20260425-007
Date: 25 April 2026
Time Opened (IST): 09:18
Severity: High
Client: FinSecure Bank
Industry: BFSI (Banking, Financial Services, Insurance)
L1 Analyst: Arjun Mehta
L2 Analyst: Rohan Desai
L3 Analyst: Nisha Kapoor
Current Status: Under Investigation
Sentinel Incident Number: 482731

Section 2 — Client & Environment Context

FinSecure Bank is a mid-sized private banking institution headquartered in Mumbai, serving retail and corporate clients across India. With approximately 1,200 employees and three regional branch offices, the bank operates a hybrid infrastructure that blends traditional on-premises systems with a growing Azure cloud footprint. Its core banking system (CBS) remains hosted within a tightly controlled on-premises data center, while customer-facing applications and analytics workloads are increasingly deployed in Azure.

The endpoint estate consists of roughly 950 Windows 10/11 workstations used by employees across departments such as finance, customer operations, and loan processing. In addition, around 80 Windows Server 2022 systems support domain services, application hosting, and internal databases. Identity is managed through an on-prem Active Directory environment synchronized with Azure AD using Azure AD Connect. All users authenticate through Azure AD, with Conditional Access policies enforcing MFA across the organization.

FinSecure operates under strict regulatory oversight, including the RBI IT Framework and PCI-DSS 4.0 compliance requirements. This makes privileged access tightly controlled and heavily audited. Global Administrators, Privileged Role Administrators, and Domain Admins represent a critical attack surface — compromise of these accounts could enable unauthorized fund transfers, manipulation of customer data, or disruption of core banking operations.

Microsoft Defender for Endpoint is deployed across all systems, providing deep telemetry into process execution and system behavior. Logs from endpoints, identity systems, and cloud services are aggregated into Microsoft Sentinel, which acts as the central SIEM. Given the bank's recent audit cycle and heightened regulatory scrutiny, any anomaly involving privileged access

— especially interactions with sensitive processes like LSASS — is treated with heightened concern.

On the morning of this incident, no major outages or security advisories were active. However, the environment had already experienced increased probing activity earlier in the week, placing the SOC team on heightened awareness for credential-focused attacks.

Section 3 — Attack Narrative

The attacker had already established a foothold inside the network hours before this moment. The initial entry point had not required sophisticated exploits — a combination of weak endpoint hygiene and credential exposure had been enough. What mattered now was escalation.

From their vantage point inside a standard user workstation, the attacker began probing the system for privilege escalation opportunities. They were not interested in noisy exploitation techniques; stealth and persistence mattered more than speed. Their objective was clear: obtain elevated privileges without triggering obvious alerts.

They started by enumerating running processes and system privileges, identifying high-value targets. One process stood out immediately — LSASS, the Local Security Authority Subsystem Service. It held what they needed: authentication tokens, credential material, and potentially reusable access tokens.

Instead of attempting direct credential dumping — which could trigger well-known detections — the attacker chose a subtler approach: token impersonation. Using a custom toolset designed to interact with Windows security tokens, they initiated a process that requested access to LSASS. The parent process they used was intentionally selected to blend in with legitimate system activity, avoiding commonly flagged binaries.

The tool executed quietly, opening a handle to LSASS with specific access rights. From there, it duplicated an existing token belonging to a higher-privileged user session that had previously logged onto the machine. This was not a brute-force escalation — it was surgical. The attacker did not need to crack passwords; they simply borrowed trust that already existed in the system.

Once the token was duplicated, they spawned a new process under the context of the elevated token. To the operating system, this process now appeared to belong to a privileged account. The attacker verified their success by querying system privileges — administrative rights were now active.

They paused briefly.

Escalation was only valuable if it remained undetected. No outbound connections were made yet. No lateral movement initiated. Instead, they prepared the next stage carefully — reconnaissance of privileged paths and potential domain-level targets.

The attacker had achieved what they came for: silent elevation without raising immediate alarms. Now the real operation could begin.

Section 4 — Detection & Alert

At 09:18 IST, a Microsoft Sentinel analytics rule titled **"Suspicious LSASS Access via Unusual Parent Process"** triggered a High-severity alert. The rule was designed to detect abnormal access attempts to the LSASS process — a critical indicator often associated with credential access or token manipulation techniques.

The rule logic focused on identifying processes attempting to access LSASS with sensitive access rights, especially when the initiating (parent) process did not belong to a known safe list of system binaries.

Sentinel Analytics Rule — KQL Logic:

```
DeviceProcessEvents
| where TimeGenerated > ago(5m)  // Scope to recent activity
| where FileName =~ "lsass.exe"  // Target process is LSASS
| join kind=inner (
   DeviceProcessEvents
   | where InitiatingProcessFileName !in~
("lsass.exe","wininit.exe","services.exe","taskmgr.exe")
   // Filter out known safe parent processes
) on DeviceId
| where InitiatingProcessCommandLine contains_any ("access", "token", "duplicate")
// Indicators of token manipulation or LSASS access attempts
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
InitiatingProcessCommandLine, AccountName
```

At **09:18:07 IST**, the rule matched activity on endpoint **FSB-WKS-447**, a workstation assigned to a finance department user.

The triggering event showed:

- **Initiating Process:** svchost.exe (unusual for LSASS interaction in this context)
- **Command Line:** contained arguments referencing token duplication behavior

- **Account:** fsb.local\rahul.singh (standard user account)
- **Access Target:** LSASS process

The anomaly lay not just in LSASS access, but in the mismatch between the initiating process and expected system behavior. svchost.exe typically hosts system services, but in this case, it was executing with parameters that suggested token manipulation — a deviation from baseline.

In the **Microsoft Sentinel Incidents queue**, the alert appeared as:

- **Title:** Suspicious LSASS Access via Unusual Parent Process
- **Severity:** High
- **Entities:**
    - Account: rahul.singh@finsecure.com
    - Host: FSB-WKS-447
    - Process: svchost.exe → lsass.exe
- **Time:** 09:18 IST

The alert surfaced near the top of Arjun's queue, highlighted in yellow against a backdrop of lower-priority incidents. No accompanying alerts had yet been correlated to this event — this was the first signal.

Section 5 — L1 Triage (Arjun Mehta)

Arjun was mid-sip of his second coffee when the incident slid into the top of his Sentinel queue.

09:18. High severity.

His eyes shifted instinctively to the left monitor — the Incidents queue. He read the title once, then again under his breath, a habit he never quite noticed himself doing.

"Suspicious LSASS access… unusual parent process…"

That was enough to click.

The right panel expanded on his secondary monitor. He leaned forward slightly, scanning the alert description. LSASS access always meant one thing — credentials or tokens. There was no harmless explanation that came easily to mind.

He moved to the **Entities** section.

- Host: FSB-WKS-447
- User: rahul.singh
- Process: svchost.exe

Arjun frowned.

“svchost?” he muttered.

That didn’t sit right. LSASS access from known tools — yes. But svchost doing token-related operations? That wasn’t something he had seen in normal baselines.

He clicked into the **Entity Behavior** tab for the device.

The timeline loaded — last 24 hours of activity. He scanned quickly, eyes jumping across process spikes and authentication events. Nothing immediately obvious stood out — no massive spikes, no repeated alerts. The system looked… quiet.

Too quiet.

He switched to **Defender for Endpoint** from his browser tab. Navigating to the device timeline for FSB-WKS-447, he filtered for process events around 09:18.

There it was.

A process chain:

- Parent: svchost.exe
- Child: a short-lived process with no standard naming pattern
- Action: handle access to lsass.exe

He clicked into the command-line details.

The string wasn’t fully clear, but words stood out:

“DuplicateToken… OpenProcess…”

Arjun leaned back slightly.

Token duplication.

He had seen this in training modules — privilege escalation without passwords. But he had never handled it live.

For a moment, he hesitated.

Could this be a legitimate system operation? Some internal service behavior?

He glanced at his notepad — a page where he had scribbled reference notes weeks ago. One line stood out:

“LSASS access + token keywords = HIGH RISK ALWAYS ESCALATE”

He exhaled quietly.

Better to escalate early than miss something like this.

Before making the call, he ran one quick check.

He opened a quick query view in Sentinel and checked if the same user had logged in with elevated privileges after this event. No immediate sign of privilege assignment events yet — but that didn’t mean escalation hadn’t occurred locally.

That uncertainty sealed it.

He clicked back into the incident and began typing into the **Sentinel Incident comments** — writing in full sentences, as always:

“Observed LSASS access from svchost.exe on FSB-WKS-447 under user rahul.singh. Command-line indicators suggest token duplication activity (keywords: DuplicateToken, OpenProcess). No prior baseline behavior observed for this device. Potential privilege escalation attempt.”

He changed nothing about severity — High was appropriate.

Then he opened **Microsoft Teams**, switching to the **SOC-Incidents** channel.

His fingers moved quickly, but deliberately.

09:23 IST — Escalation Message:

[INC-20260425-007 | HIGH]
Alert triggered for suspicious LSASS access on host FSB-WKS-447 (user: rahul.singh).
Initiating process: svchost.exe with command-line indicators of token duplication (DuplicateToken/OpenProcess observed).
No similar baseline behavior for this host. Potential privilege escalation via token impersonation.
No confirmed lateral movement or post-escalation activity yet.
Requesting L2 to validate LSASS access context, confirm token abuse, and check for privilege elevation or follow-on actions.

He hit send.

Then he pinged Rohan directly.

And waited — eyes already drifting back to the process tree.

Section 6 — L2 Investigation (Rohan Desai)

Rohan didn't open Sentinel immediately.

At 09:24 IST, his eyes were on the Teams notification first.

He read Arjun's escalation message once, then again — slower this time.

"Token duplication… svchost… no baseline…"

He nodded slightly.

Good catch.

Only then did he turn to his center monitor and open the incident.

The **Sentinel investigation graph** expanded across the screen — a visual cluster of the workstation, user account, and process chain. Nothing sprawling yet. This was still early-stage.

That meant one thing: reconstruct before it spreads.

He opened **Notepad++** on his second monitor — his KQL scratchpad already filled with fragments from previous incidents. He added a new section at the bottom.

Question 1: *Did the user gain elevated privileges after LSASS access?*

He began writing the query.

```
SecurityEvent
| where TimeGenerated between (datetime(2026-04-25 09:10:00) .. datetime(2026-04-25 09:30:00))
// Focus on time window around alert
| where EventID == 4672
// Special privileges assigned to new logon
| where Account contains "rahul.singh"
// Filter for the affected user
| project TimeGenerated, Computer, Account, PrivilegeList
```

He ran it.

Result: **1 event at 09:19:12 IST**

Privileges assigned included:

- SeDebugPrivilege
- SeImpersonatePrivilege

Rohan leaned back slightly.

“That’s not normal for a standard user.”

This confirmed it — token impersonation had worked.

He added a note in his scratchpad.

Confirmed privilege escalation post-LSASS access.

Next question.

Question 2: *What process chain led to LSASS access?*

```
DeviceProcessEvents
| where TimeGenerated between (datetime(2026-04-25 09:00:00) .. datetime(2026-04-25 09:20:00))
// Capture process activity leading up to alert
| where DeviceName == "FSB-WKS-447"
| where InitiatingProcessFileName =~ "svchost.exe"
| project TimeGenerated, FileName, InitiatingProcessFileName, ProcessCommandLine, InitiatingProcessCommandLine
```

The results told a clearer story.

At **09:17:52 IST**:

- svchost.exe launched a transient process
- Command line included token manipulation flags
- Followed immediately by LSASS handle access

Rohan zoomed into the process lineage.

No legitimate service tag. No standard service group.

“This isn’t normal svchost behavior,” he muttered.

He pivoted.

Question 3: *What happened BEFORE this? How did attacker get here?*

He extended the timeline back two hours — querying process execution and authentication logs.

He identified an earlier suspicious process:

- Unknown executable launched from user temp directory at 07:41 IST
- No digital signature
- Spawned under same user account

That was likely the entry point.

Now he began building the timeline.

Attack Timeline

Timestamp (IST)	Event	Data Source	MITRE Technique
07:41:22	Unknown executable launched from temp directory	DeviceProcessEvents	T1059 (Execution)
09:17:52	svchost.exe spawns process with token manipulation flags	DeviceProcessEvents	T1134 (Token Manipulation)
09:18:07	LSASS access initiated	DeviceProcessEvents	T1003 (Credential Access)
09:19:12	Special privileges assigned to user	SecurityEvent	T1134
09:23:00	L1 escalation	Sentinel	—

Now IOCs.

He extracted everything.

IOC Table

IOC	Type	Source	Enrichment Result

FSB-WKS-447	Host	DeviceProcessEvents	Internal asset
rahul.singh	User	SecurityEvent	Standard user
svchost.exe (abnormal invocation)	Process	DeviceProcessEvents	Legit binary, suspicious usage
Unknown.exe (temp path)	File	DeviceProcessEvents	No signature, not in VirusTotal
—	—	—	—

He checked VirusTotal for the hash of the unknown executable.

Result: No hits.

Custom or newly compiled.

That raised the stakes.

MITRE ATT&CK Mapping

- T1059 — Execution
- T1134 — Access Token Manipulation
- T1003 — OS Credential Dumping (LSASS access)

Now containment.

He didn't write "isolate device."

He was more precise.

Containment Recommendations

1. **Isolate endpoint FSB-WKS-447** using Microsoft Defender for Endpoint "Isolate Device" action immediately.
2. **Force password reset** for user rahul.singh and invalidate all active sessions via Azure AD.
3. **Revoke all tokens** for the account using Azure AD sign-out everywhere control.
4. **Block execution from user temp directories** via Defender Attack Surface Reduction rule.

5. **Hunt for similar LSASS access patterns** across all endpoints using the same query logic.
6. **Check for lateral movement attempts** from FSB-WKS-447 to internal servers in last 2 hours.

Rohan sat back.

Clean escalation. Clear chain.

But one thing lingered.

No outbound traffic yet.

Which meant—

“They haven’t moved yet.”

He saved his notes.

And started preparing for the next phase.

Section 7 — L3 Detection Tuning (Nisha Kapoor)

At 11:10 IST, Nisha was already at her desk, a fresh cup of tea sitting untouched to her right. Rohan’s investigation report was open on her ultra-wide monitor, the timeline section centered on screen. She read slowly, not scanning — absorbing.

07:41 — initial execution.
09:17 — token manipulation.
09:18 — LSASS access.

Her eyes paused there.

“That’s late,” she said quietly.

The first detection had only triggered at the LSASS access stage — well after the attacker had already executed code and begun privilege escalation. That meant the rule had worked… but not early enough.

She shifted to the **Microsoft Sentinel Analytics Rules editor** and pulled up the rule:

“Suspicious LSASS Access via Unusual Parent Process”

She read the KQL carefully, line by line.

Before-State KQL Rule

```
DeviceProcessEvents
| where TimeGenerated > ago(5m)
| where FileName =~ "lsass.exe"
| join kind=inner (
   DeviceProcessEvents
   | where InitiatingProcessFileName !in~
("lsass.exe","wininit.exe","services.exe","taskmgr.exe")
) on DeviceId
| where InitiatingProcessCommandLine contains_any ("access", "token", "duplicate")
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
InitiatingProcessCommandLine, AccountName
```

She leaned back slightly.

The rule was narrow — too narrow.

It depended heavily on **specific keywords in the command line**: “access”, “token”, “duplicate”. That worked in this case because the attacker’s tool exposed those strings. But what if they hadn’t?

She opened Notepad++ and wrote a single sentence at the top of her draft:

Identified Weakness:
The rule relies on command-line keyword matching, making it easy to evade with obfuscation or alternate tooling that does not expose these strings.

She turned back to the rule editor.

What mattered wasn’t the words — it was the behavior.

LSASS access from unusual processes. That was the invariant.

She began rewriting.

After-State Improved KQL Rule

```
DeviceProcessEvents
| where TimeGenerated > ago(5m)
```

```
// Scope to recent activity
| where FileName =~ "lsass.exe"
// Target process is LSASS

| join kind=inner (
   DeviceProcessEvents
   | where InitiatingProcessFileName !in~
("lsass.exe","wininit.exe","services.exe","taskmgr.exe","lsm.exe")
   // Exclude known legitimate LSASS-accessing processes
) on DeviceId

| where InitiatingProcessIntegrityLevel !~ "System"
// Focus on non-system processes attempting LSASS access

| where InitiatingProcessFileName !startswith "MsMpEng"
// Exclude Defender AV legitimate access patterns

| summarize count() by DeviceName, InitiatingProcessFileName, AccountName,
bin(TimeGenerated, 1m)
// Aggregate repeated attempts within short window

| where count_ >= 1
// Even a single abnormal LSASS access is high signal

| project TimeGenerated, DeviceName, InitiatingProcessFileName, AccountName
```

She read it again, this time imagining the attacker.

No keywords needed. No reliance on command-line visibility.

Just behavior.

She added a comment at the bottom of her draft:

“This rule shifts detection from string-based indicators to behavioral detection of LSASS access from non-system, non-approved processes. It reduces evasion risk and ensures earlier detection regardless of tool implementation.”

She glanced back at the timeline.

If this version had been active, the detection would still have fired at LSASS access — but it would have been **more reliable across variations**, and could be extended further in the future to correlate with pre-LSASS execution.

Nisha finally reached for her tea.

“Next step,” she murmured, “tie this with pre-execution signals.”

But that would be another rule.

Section 8 — Final Incident Report

Part A — Executive Summary

On 25 April 2026, FinSecure Bank experienced a security incident involving unauthorized privilege escalation on an employee workstation. An attacker executed a malicious process that accessed sensitive system components and elevated privileges without using valid credentials. The activity was detected by security monitoring tools and escalated promptly. The affected system was isolated, and the user account was secured before any further movement or data access occurred. No evidence of data theft or system-wide compromise was found. Immediate containment actions were successful, and additional detection improvements have been implemented to strengthen early identification of similar threats.

Part B — Full Technical Summary

Incident ID: INC-20260425-007
Client: FinSecure Bank
Date: 25 April 2026

Attack Type

Privilege Escalation via Access Token Manipulation

MITRE ATT&CK Techniques

- T1059 — Command Execution
- T1134 — Access Token Manipulation

- T1003 — OS Credential Dumping (LSASS Access)

Timeline Summary

Time (IST)	Event Description
07:41	Suspicious executable launched from user temp directory
09:17	svchost.exe spawned process performing token manipulation
09:18	LSASS access detected
09:19	Elevated privileges assigned to user session
09:23	Incident escalated to L2
09:40	Endpoint isolated and account secured

Impact Assessment

- No evidence of lateral movement observed
- No data exfiltration detected
- No persistence mechanisms confirmed
- Impact contained to single endpoint (FSB-WKS-447)
- Privilege escalation was successful but short-lived due to rapid detection

---

IOC List

IOC	Type	Enrichment Result
FSB-WKS-447	Host	Internal corporate endpoint
rahul.singh@finsecure.com	User	Valid internal user account
svchost.exe (abnormal behavior)	Process	Legitimate binary misused
Unknown.exe (temp directory)	File	No signature, no VirusTotal hits

---

Containment Actions Taken

- Endpoint FSB-WKS-447 isolated via Defender for Endpoint
- User account password reset and sessions revoked
- Token invalidation performed in Azure AD
- Suspicious executable removed and quarantined
- Environment-wide scan initiated for similar indicators

Lessons Learned

- LSASS access remains a high-confidence indicator of credential abuse
- Command-line dependent detections are susceptible to evasion
- Early-stage execution activity (temp directory binaries) requires stronger detection coverage
- Privileged escalation can occur rapidly without triggering multiple alerts

---

Action Items

Action	Owner	Priority	Target Date
Deploy improved LSASS behavioral detection rule	Nisha Kapoor (L3)	High	28-Apr-2026
Implement ASR rule to block execution from temp directories	Endpoint Security Team	High	30-Apr-2026
Conduct environment-wide hunt for similar token manipulation patterns	Rohan Desai (L2)	High	27-Apr-2026
Review privilege escalation monitoring coverage across endpoints	SOC Engineering Team	Medium	05-May-2026
Conduct user awareness training on suspicious activity reporting	Security Awareness Team	Medium	15-May-2026

Incident Status: Contained and Closed
Report Prepared By: SOC Team — FinSecure MSSP
Reviewed By: Nisha Kapoor (L3 Detection Engineering)